

UNIVERSITÉ CHEIKH ANTA DIOP DE DAKAR

INSTITUT CONFUCIUS

Année académique: 2025-2026

PROJET DE FIN D'ÉTUDES

Thème :

Conception et développement d'un système d'administration Linux automatisé

Présenté par :

ABDOU RAHIM NDIAYE

Module : Programmation Linux

Professeure : M. DIOP

1. RÉSUMÉ

Ce projet a pour objectif de concevoir et développer un système complet d'administration Linux automatisé. Il comporte deux parties complémentaires : une partie en Shell Bash qui propose un menu interactif permettant de gérer les répertoires, les fichiers, les permissions, les processus, la surveillance du système et la planification de tâches ; et une partie en langage C qui implémente un programme de gestion d'inventaire informatique. Les deux parties fonctionnent ensemble sous Linux. Le projet utilise les outils gcc pour la compilation, make pour l'automatisation de la compilation, gdb pour le débogage et git pour la gestion des versions. L'ensemble du code source est disponible sur GitHub.

2. INTRODUCTION

L'administration système Linux est une activité quotidienne qui demande la réalisation de nombreuses opérations répétitives : création et gestion de répertoires et de fichiers, surveillance des ressources système, gestion des processus, sauvegarde des données et planification de tâches automatiques.

L'automatisation de ces opérations permet de gagner du temps, de réduire les erreurs humaines et d'améliorer la fiabilité du système. C'est dans ce contexte que s'inscrit ce projet, qui vise à développer une application complète permettant d'automatiser ces opérations à l'aide des outils étudiés durant le module de Programmation Linux.

Les objectifs du projet sont :

- Créer un menu interactif Bash couvrant toutes les opérations d'administration
- Développer un programme C de gestion d'inventaire informatique
- Utiliser les bonnes pratiques de développement : Makefile, gdb, git

3. PRÉSENTATION DE LINUX

Linux est un système d'exploitation libre et open source créé par Linus Torvalds en 1991. Il est basé sur le noyau Linux et est largement utilisé dans les serveurs, les systèmes embarqués et les postes de travail.

Linux est particulièrement apprécié en administration système pour plusieurs raisons :

- Il est stable et sécurisé
- Il offre un contrôle total sur le système via le terminal
- Il propose des outils puissants comme bash, cron, ps, grep, find
- Il est gratuit et dispose d'une large communauté

Dans ce projet, nous utilisons Ubuntu 24.04 via WSL (Windows Subsystem for Linux), qui permet d'exécuter Linux directement sous Windows sans machine virtuelle.

4. ANALYSE DES BESOINS

Besoins fonctionnels

Partie Bash :

- Gestion des répertoires : créer, supprimer, renommer, afficher
- Gestion des fichiers : créer, copier, déplacer, supprimer, rechercher
- Gestion des permissions : chmod, chown, affichage
- Recherche : fichier, texte, utilisateur, processus
- Surveillance : mémoire, disque, utilisateurs connectés, CPU
- Gestion des processus : afficher, rechercher, arrêter, signaler
- Sauvegarde : sauvegarder, compresser, dater, restaurer
- Planification : cron pour les tâches récurrentes, at pour les tâches ponctuelles

Partie C :

- Gérer un inventaire informatique avec les champs : ID, nom, IP, MAC, localisation, date
- Permettre : ajouter, modifier, supprimer, rechercher, afficher, sauvegarder

Besoins techniques

- Compilation avec gcc et Makefile
- Débogage avec gdb
- Gestion de versions avec git et GitHub

5. CONCEPTION

Structure du projet

ProjetLinux/

```
├─ README.md      → Documentation
├─ Makefile       → Compilation automatique
├─ menu.sh        → Menu Bash interactif
├─ sauvegarde.sh  → Script de sauvegarde
├─ install.sh     → Script d'installation
├─ src/
|   ├── main.c    → Programme principal C
|   ├── equipement.c → Fonctions gestion inventaire
|   ├── equipement.h → Déclarations structure et fonctions
|   ├── fichier.c  → Fonctions copie et lecture fichiers
|   └─ fichier.h  → Déclarations fichier.c
├─ data/
|   └─ equipments.txt → Base de données
├─ backup/        → Sauvegardes automatiques
└─ docs/
```

└─ rapport.pdf → Ce rapport

Structure de la struct Equipement (C

```
typedef struct {  
    int id;           // Identifiant unique  
    char nom[50];     // Nom de l'équipement  
    char ip[20];      // Adresse IP  
    char mac[20];     // Adresse MAC  
    char localisation[50]; // Lieu physique  
    char date_ajout[20]; // Date d'ajout automatique  
} Equipement;
```

Architecture du menu Bash

Le menu principal propose 8 sous-menus. Chaque sous-menu est une fonction bash indépendante qui s'exécute en boucle jusqu'au choix "Retour". Le menu utilise la commande case pour traiter les choix de l'utilisateur.

6. DESCRIPTION DES SCRIPTS BASH

menu.sh

C'est le fichier principal de la partie Bash. Il contient 9 fonctions :

- menu_principal() : affiche le menu général avec 8 options et dirige vers le sous-menu choisi
- menu_repertoires() : permet de créer, supprimer, renommer et afficher des dossiers avec les commandes mkdir, rm -rf, mv, ls
- menu_fichiers() : gère les fichiers avec touch, cp, mv, rm, find
- menu_permissions() : affiche et modifie les permissions avec ls -la, chmod, chown
- menu_recherche() : recherche avec find pour les fichiers, grep pour les textes, ps aux pour les processus
- menu_surveillance() : affiche l'état du système avec free -h, df -h, who, top
- menu_processus() : liste et contrôle les processus avec ps aux, kill
- menu_sauvegarde() : appelle sauvegarde.sh et permet la restauration avec tar -xzf
- menu_planification() : programme des tâches avec crontab et at

sauvegarde.sh

Script de sauvegarde automatique qui :

- Prend un répertoire en argument
- Crée une archive .tar.gz avec la date dans le nom
- Stocke la sauvegarde dans ~/ProjetLinux/backup/
- Affiche la taille du fichier créé

7. DESCRIPTION DU PROGRAMME C

main.c

Point d'entrée du programme. Charge les données au démarrage avec charger_equipements(), affiche le menu en boucle et appelle la fonction correspondante selon le choix.

equipement.h

Fichier d'en-tête qui déclare la structure Equipement et les prototypes de toutes les fonctions. Il utilise des gardes d'inclusion (#ifndef) pour éviter les inclusions multiples.

equipement.c

Contient l'implémentation de toutes les fonctions de gestion :

- charger_equipements() : lit le fichier data/equipements.txt avec fscanf et charge les données dans un tableau global
- sauvegarder_equipements() : écrit tous les équipements dans le fichier avec fprintf
- ajouter_equipement() : saisit les informations, génère la date automatiquement avec time() et strftime()
- afficher_equipements() : affiche un tableau formaté avec printf
- rechercher_equipement() : utilise strstr() pour chercher dans nom, IP et MAC
- modifier_equipement() : trouve l'équipement par ID et modifie ses champs
- supprimer_equipement() : supprime en décalant les éléments du tableau

fichier.c

Contient deux fonctions utilitaires :

- copier_fichier() : copie caractère par caractère avec fgetc et fputc
- afficher_fichier() : lit et affiche ligne par ligne avec fgets

Makefile

Automatise la compilation avec les règles :

- make → compile tout
- make clean → supprime les fichiers compilés
- make run → compile et lance le programme

8. DIFFICULTÉS RENCONTRÉES

Durant la réalisation de ce projet, plusieurs difficultés ont été rencontrées :

Installation de gcc : lors de la première tentative d'installation, des erreurs 404 sont apparues à cause de problèmes de connexion aux serveurs Ubuntu. Le problème a été résolu en utilisant `sudo apt --fix-missing install -y gcc`.

Fichiers d'en-tête manquants : après installation de gcc, le compilateur ne trouvait pas `stdio.h`. L'installation de `libc6-dev` a résolu le problème.

Le Makefile : les règles du Makefile doivent obligatoirement commencer par une tabulation (Tab) et non des espaces, sinon make affiche une erreur. Cette contrainte a nécessité une attention particulière.

La gestion des chaînes en C : la lecture de chaînes avec espaces avec `scanf` nécessite le format `%[^\n]` au lieu de `%s` pour capturer les espaces dans les noms et localisations.

9. CAPTURES D'ÉCRAN

```
=====
      SYSTÈME D'ADMINISTRATION LINUX
      Institut Confucius – UCAD
=====
1. Gestion des répertoires
2. Gestion des fichiers
3. Gestion des permissions
4. Recherche
5. Surveillance du système
6. Gestion des processus
7. Sauvegarde automatique
8. Planification
0. Quitter
=====
Votre choix : █
```

Figure 1 : Menu principal du Script Bash

```
=== SURVEILLANCE DU SYSTÈME ===
1. Mémoire utilisée
2. Espace disque
3. Utilisateurs connectés
4. Charge processeur
0. Retour
Votre choix : 1
              total        used        free      shared  buff/cache   available
Mem:          7.7Gi        1.6Gi        6.0Gi        3.7Mi        292Mi        6.1Gi
Swap:          2.0Gi          0B         2.0Gi
Entrée pour continuer... █
```

Figure 2 : Surveillance de la mémoire avec free-h

```
gcc -Wall -g -c src/main.c -o src/main.o
gcc -Wall -g -c src/equipement.c -o src/equipement.o
gcc -Wall -g -c src/fichier.c -o src/fichier.o
gcc -Wall -g -o inventaire src/main.o src/equipement.o src/fichier.o
Compilation reussie : ./inventaire █
```

Figure 3 : Compilation du programme C avec make

```
Nom de l'equipement : PC-Salle-Info-01
Adresse IP : 192.168.1.10
Adresse MAC : AA:BB:CC:DD:EE:01
Localisation : Salle Informatique █
```

Figure 4 : Ajout d'un équipement dans l'inventaire C

ID	Nom	IP	MAC	Localisation	Date
1	0	0	0	0	2026-07-18
2	PC-Salle-Info-01	192.168.1.10	AA:BB:CC:DD:EE:01	Salle Informatique	2026-07-18
3	Switch-Cisco-01	192.168.1.1	BB:CC:DD:EE:FF:02	Salle-Serveur	2026-07-18
Total : 3 equipement(s).					

Figure 5 :Affichage de tous les équipements en tableau

```

(gdb) break main
Breakpoint 1 at 0x1437: file src/main.c, line 31.
(gdb) run
Starting program: /home/rahim/ProjetLinux/inventaire
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at src/main.c:31
31     int main() {
(gdb) next
34     printf("\nChargement de l'inventaire...\n");
(gdb) quit

```

Figure 6 : Debugage avec gdb-execution pas a pas

```

53f4a6d (HEAD -> main) Ajout rapport PDF et finalisation du projet
38ea619 (origin/main) Projet complet : menu Bash + inventaire C + Makefile

```

Figure 7 : Historique des commits git

```

Sauvegarde de '/home/rahim/ProjetLinux' en cours...
Sauvegarde créée : /home/rahim/ProjetLinux/backup/sauvegarde_2026-07-18_23-14-29.tar.gz
Taille : 72K
rahim@DESKTOP-DKFD39V:~/ProjetLinux$ ls ~/ProjetLinux/backup/
sauvegarde_2026-07-18_23-14-29.tar.gz

```

Figure 8 : Sauvegarde automatique avec date dans le nom

10. CONCLUSION

Ce projet nous a permis de mettre en pratique les connaissances acquises durant le module de Programmation Linux. Nous avons développé un système complet qui combine la puissance des scripts Bash pour l'administration système et le langage C pour la gestion de données structurées.

La réalisation de ce projet a approfondi notre maîtrise des commandes Linux fondamentales, de la programmation en C avec fichiers et structures, des outils de développement professionnels comme gcc, make, gdb et git, et de l'automatisation des tâches avec cron et at.

Ce type de système est directement applicable dans un environnement professionnel d'administration système, ce qui renforce l'intérêt pédagogique de ce projet.

11. BIBLIOGRAPHIE

Cours de M. DIOP — Module Programmation Linux, Institut Confucius UCAD 2025-2026

Pages de manuel Linux : man bash, man gcc, man make, man gdb, man crontab, man at

- Documentation GNU Bash : <https://www.gnu.org/software/bash/manual/>
- Documentation GCC : <https://gcc.gnu.org/onlinedocs/>
- Pro Git Book : <https://git-scm.com/book/fr/v2>

